🔥 Perfect choice — **DRF + JWT + Google Login** = **modern, production-grade auth stack**
এটা আমি তোমাকে **clean architecture + security mindset** দিয়ে বুঝাবো।

> Use case
> 👉 React / Mobile frontend
> 👉 Django REST backend
> 👉 Google login
> 👉 JWT access + refresh token

🔐 DRF + JWT + Google Login (Complete Guide)

🧠 Final Architecture (Big Picture)

```
Frontend (React / Mobile)
 └── Google Login (OAuth)
       └── Google ID Token
             ↓
Backend (Django + DRF)
 └── Verify Google token
 └── Create / Get User
 └── Issue JWT (access + refresh)
             ↓
Frontend stores JWT
```

1️⃣ Required Packages

```
pip install djangorestframework
pip install djangorestframework-simplejwt
pip install google-auth
```

2️⃣ settings.py (DRF + JWT Setup)

🔹 INSTALLED_APPS

```
INSTALLED_APPS = [
    'rest_framework',
```

```
    'accounts',   # your app
]
```

🔹 REST_FRAMEWORK

```
from datetime import timedelta

REST_FRAMEWORK = {
    'DEFAULT_AUTHENTICATION_CLASSES': (
        'rest_framework_simplejwt.authentication.JWTAuthentication',
    ),
}
```

🔹 SIMPLE_JWT (Security-focused)

```
SIMPLE_JWT = {
    'ACCESS_TOKEN_LIFETIME': timedelta(minutes=5),
    'REFRESH_TOKEN_LIFETIME': timedelta(days=7),
    'ROTATE_REFRESH_TOKENS': True,
    'BLACKLIST_AFTER_ROTATION': True,
    'AUTH_HEADER_TYPES': ('Bearer',),
}
```

3️⃣ URLs (JWT Endpoints)

```
from rest_framework_simplejwt.views import (
    TokenObtainPairView,
    TokenRefreshView
)

urlpatterns = [
    path('api/token/', TokenObtainPairView.as_view()),
    path('api/token/refresh/', TokenRefreshView.as_view()),
]
```

(Username/password login — optional)

4️⃣ Google OAuth (Frontend Part – Quick Idea)

Frontend uses:

- Google Identity Services
- Gets **id_token**

Example response:

```
{
  "credential": "eyJhbGciOiJSUzI1NiIsImtpZCI6..."
}
```

👉 This `credential` = **Google ID Token**

5️⃣ Google Login API (Backend Core Logic)

🔹 views.py

```
from rest_framework.views import APIView
from rest_framework.response import Response
from rest_framework import status
from google.oauth2 import id_token
from google.auth.transport import requests
from django.conf import settings
from django.contrib.auth import get_user_model
from rest_framework_simplejwt.tokens import RefreshToken

User = get_user_model()

class GoogleLoginAPIView(APIView):
    def post(self, request):
        token = request.data.get('token')

        if not token:
            return Response(
                {"error": "Token missing"},
                status=status.HTTP_400_BAD_REQUEST
            )

        try:
            idinfo = id_token.verify_oauth2_token(
                token,
                requests.Request(),
                settings.GOOGLE_CLIENT_ID
            )
```

```
            email = idinfo['email']
            name = idinfo.get('name', '')

            user, created = User.objects.get_or_create(
                email=email,
                defaults={'name': name}
            )

            refresh = RefreshToken.for_user(user)

            return Response({
                "access": str(refresh.access_token),
                "refresh": str(refresh),
                "user": {
                    "email": user.email,
                    "name": user.name
                }
            })

        except ValueError:
            return Response(
                {"error": "Invalid Google token"},
                status=status.HTTP_400_BAD_REQUEST
            )
```

6️⃣ settings.py (Google Client ID)

```
GOOGLE_CLIENT_ID = "YOUR_GOOGLE_CLIENT_ID.apps.googleusercontent.com"
```

⚠️ Never commit this in Git
Use `.env` in production

7️⃣ URL Route

```
path('api/auth/google/', GoogleLoginAPIView.as_view()),
```

8️⃣ Protect Any API with JWT

```python
from rest_framework.permissions import IsAuthenticated

class ProfileAPIView(APIView):
    permission_classes = [IsAuthenticated]

    def get(self, request):
        return Response({
            "email": request.user.email,
            "name": request.user.name
        })
```

Frontend header:

```
Authorization: Bearer ACCESS_TOKEN
```

🔐 Security MUST-HAVES (Read Carefully)

✅ Verify audience (already done)

```
verify_oauth2_token(token, ..., GOOGLE_CLIENT_ID)
```

✅ Verified email only

```python
if not idinfo.get('email_verified'):
    raise ValueError("Email not verified")
```

✅ HTTPS only (production)

```
CSRF_COOKIE_SECURE = True
SESSION_COOKIE_SECURE = True
```

9️⃣ Refresh Token Flow (Frontend)

```
POST /api/token/refresh/
{
  "refresh": "REFRESH_TOKEN"
}
```

Returns new access token.

1️⃣0️⃣ Common Mistakes 🚨

❌ Using Google access_token instead of id_token
❌ Not verifying client_id
❌ Storing JWT in localStorage (XSS risk)
❌ Long access token lifetime
❌ No refresh rotation

1️⃣1️⃣ Recommended Token Storage

Platform	Storage
Web	HttpOnly cookie
Mobile	Secure Storage
SPA	Memory / cookie

1️⃣2️⃣ Real-World Improvements (🔥)

- Assign role on first Google login
- Block unknown email domains
- Save Google profile picture
- Logout = blacklist refresh token
- Device-based refresh token

🧠 Final Verdict

✅ **This is the industry-standard approach**
✅ Used by startups + big companies
✅ Scales perfectly for ML / AI systems

If you want next:
👉 **React Google Login + Django JWT (full code)**
👉 **Google login + role-based access**
👉 **Google login + multiple devices**
👉 **Deploy-ready security checklist**

Just say the word 😎

Got it 👍
তুমি এখন **Google Cloud Dashboard**-এ আছো, তাই “secret key” **দেখা যাচ্ছে না**।
ওটা এখান থেকে না — **Credentials page** থেকে আসে।

চল exact click-by-click বলি (তোমার screenshot অনুযায়ী 👇)

🔑 allauth এর Google Client ID & Secret কোথায় পাবা

👉 Step 1: Left sidebar → APIs & Services

(তোমার screenshot-এর বাঁ পাশে আছে)

➡️ Click **APIs & Services**

👉 Step 2: Credentials

```
APIs & Services
   └── Credentials   ✅
```

এখানেই সব OAuth key থাকে
(এই জায়গায় না এলে “no find” হবেই)

👉 Step 3: Create Credentials

Top bar এ:

```
➕ CREATE CREDENTIALS
   └── OAuth client ID
```

👉 Step 4: OAuth Consent Screen (যদি আগে না করা থাকে)

যদি বলে consent screen setup করতে:

- User type: **External**
- App name: anything

- Email: তোমার email
- Scopes: `email`, `profile`
- Save

তারপর আবার **Credentials** এ ফিরো

👉 Step 5: OAuth Client Create

Fill like this 👇

Field	Value
Application type	**Web application**
Name	django-allauth
Authorized redirect URI	`http://localhost:8000/accounts/google/login/callback/`

⚠️ **Slash সহ exact URI** দিতে হবে

👉 Step 6: Create 🎉

এখন popup আসবে যেখানে থাকবে:

- ✅ **Client ID**
- ✅ **Client Secret** ← 🔥 এইটাই allauth এর secret key

🔧 Django allauth এ কোথায় বসাবে?

Django Admin →

```
Social Applications → Add
```

Field	What to paste
Client ID	Google Client ID
Secret key	Google Client Secret
Provider	Google
Sites	example.com

Save ✔

❌ Common reason “no find” হওয়ার

❌ Firebase console দেখছো
❌ APIs page এ আছো, Credentials এ যাওনি
❌ OAuth client তৈরি করোনি
❌ Android / iOS app type নিয়েছো (Web না)

🧠 One-line rule (মনে রাখো)

> **django-allauth → Google Cloud → APIs & Services → Credentials → OAuth Client (Web)**

চাও তো next আমি:

- তোমার **exact project (phi-mart)** অনুযায়ী
- কোন button ক্লিক করবে screenshot-wise বলে দিতে পারি
- অথবা **Firebase + allauth একসাথে safe কিনা** explain করতে পারি

বললেই চলি 😄

`apply all Auth in rest_framework`

https://docs.allauth.org/en/latest/installation/quickstart.html

`get OAuth Credientials from Goole Cloude Console`

Got it 👍
তুমি এখন **Google Cloud Dashboard**-এ আছো, তাই “secret key” **দেখা যাচ্ছে না**।
ওটা এখান থেকে না — **Credentials page** থেকে আসে।

চল exact click-by-click বলি (তোমার screenshot অনুযায়ী 👇)

🔑 allauth এর Google Client ID & Secret কোথায় পাবা

👉 Step 1: Left sidebar → APIs & Services

(তোমার screenshot-এর বাঁ পাশে আছে)

➡️ Click **APIs & Services**

👉 Step 2: Credentials

```
APIs & Services
   └── Credentials   ✅
```

এখানেই সব OAuth key থাকে
(এই জায়গায় না এলে “no find” হবেই)

👉 Step 3: Create Credentials

Top bar এ:

```
➕ CREATE CREDENTIALS
   └── OAuth client ID
```

👉 Step 4: OAuth Consent Screen (যদি আগে না করা থাকে)

যদি বলে consent screen setup করতে:

- User type: **External**
- App name: anything
- Email: তোমার email
- Scopes: `email`, `profile`
- Save

তারপর আবার **Credentials** এ ফিরো

👉 Step 5: OAuth Client Create

Fill like this 👇

Field	Value
Application type	**Web application**
Name	django-allauth

Field	Value
Authorized redirect URI	`http://localhost:8000/accounts/google/login/callback/`

⚠️ **Slash সহ exact URI** দিতে হবে

👉 Step 6: Create 🎉

এখন popup আসবে যেখানে থাকবে:

- ✅ **Client ID**
- ✅ **Client Secret** ← 🔥 এইটাই allauth এর secret key

🔧 Django allauth এ কোথায় বসাবে?

Django Admin →

```
Social Applications → Add
```

Field	What to paste
Client ID	Google Client ID
Secret key	Google Client Secret
Provider	Google
Sites	example.com

Save ✔

❌ Common reason "no find" হওয়ার

❌ Firebase console দেখছো
❌ APIs page এ আছো, Credentials এ যাওনি
❌ OAuth client তৈরি করোনি
❌ Android / iOS app type নিয়েছো (Web না)

🧠 One-line rule (মনে রাখো)

> **django-allauth → Google Cloud → APIs & Services → Credentials → OAuth Client (Web)**

চাও তো next আমি:

- তোমার **exact project (phi-mart)** অনুযায়ী
- কোন button ক্লিক করবে screenshot-wise বলে দিতে পারি
- অথবা **Firebase + allauth একসাথে safe কিনা** explain করতে পারি

বললেই চলি 😄

---------Perfect 👍
আমি তোমাকে **production-ready React component** দিচ্ছি যেটা:

- ✅ Google Sign-In করে
- ✅ Google **ID token** নেয়
- ✅ DRF backend-এ পাঠায়
- ✅ Backend থেকে **JWT (access + refresh)** নেয়

👉 **Google Identity Services (official & recommended)**
(Firebase ছাড়াও কাজ করবে)

🔐 React Google Authentication Component

1️⃣ Install Google OAuth package

```
npm install @react-oauth/google axios
```

2️⃣ Wrap App with GoogleOAuthProvider

`main.jsx` / `index.jsx`

```
import React from "react";
import ReactDOM from "react-dom/client";
import App from "./App";
import { GoogleOAuthProvider } from "@react-oauth/google";

ReactDOM.createRoot(document.getElementById("root")).render(
  <GoogleOAuthProvider clientId="YOUR_GOOGLE_CLIENT_ID">
    <App />
```

```
  </GoogleOAuthProvider>
);
```

📌 **Client ID**
→ Google Cloud Console → OAuth Client (Web)

3️⃣ Google Login Component (Core Part)

`GoogleLoginButton.jsx`

```
import { GoogleLogin } from "@react-oauth/google";
import axios from "axios";

const GoogleLoginButton = () => {
  const handleSuccess = async (credentialResponse) => {
    try {
      const googleToken = credentialResponse.credential;

      const res = await axios.post(
        "http://localhost:8000/api/auth/google/",
        {
          token: googleToken,
        }
      );

      console.log("JWT:", res.data);

      // Store tokens (example)
      localStorage.setItem("access", res.data.access);
      localStorage.setItem("refresh", res.data.refresh);

    } catch (err) {
      console.error("Login failed", err);
    }
  };

  return (
    <GoogleLogin
      onSuccess={handleSuccess}
      onError={() => console.log("Login Failed")}
    />
  );
};

export default GoogleLoginButton;
```

4️⃣ Use Component

`App.jsx`

```
import GoogleLoginButton from "./GoogleLoginButton";

function App() {
  return (
    <div>
      <h2>Login with Google</h2>
      <GoogleLoginButton />
    </div>
  );
}

export default App;
```

5️⃣ DRF Backend Expectation (Reminder)

Your DRF endpoint should expect:

```
{
  "token": "GOOGLE_ID_TOKEN"
}
```

And respond:

```
{
  "access": "JWT_ACCESS",
  "refresh": "JWT_REFRESH"
}
```

(যেটা আমরা আগেই লিখেছি)

🔐 Security Best Practices (Important)

❌ Avoid

```
localStorage (XSS risk)
```

✅ Better

- HttpOnly cookies (best)
- In-memory store (ok)
- Secure mobile storage

6️⃣ Protect API Calls (Example)

```
axios.get("http://localhost:8000/api/profile/", {
  headers: {
    Authorization: `Bearer ${localStorage.getItem("access")}`,
  },
});
```

🚨 Common Mistakes

❌ Using **access_token** instead of `credential`
❌ Wrong Google Client ID
❌ Backend audience verify missing
❌ Mixing Firebase token with Google OAuth

🔥 Variations (If you want next)

- ✅ React + **Firebase Auth** version
- ✅ JWT via **HttpOnly cookies**
- ✅ Google login + **Role assignment**
- ✅ Auto refresh token interceptor
- ✅ Logout + blacklist

বললেই আমি next level version বানিয়ে দেব 😎

Short answer: **হ্যাঁ, production এ গেলে অবশ্যই URL change করতে হবে** ✅
এটা optional না — **mandatory**।

চল পরিষ্কারভাবে বুঝাই 👇

🔁 Redirect URI — dev vs production

🧪 Development (local)

```
http://localhost:8000/accounts/google/login/callback/
```

🚀 Production (example)

```
https://yourdomain.com/accounts/google/login/callback/
```

🔐 **Rules:**

- `http` → `https` (must)
- Domain exact match
- Trailing slash `/` same থাকতে হবে

🔑 কোথায় change করতে হবে?

1️⃣ Google Cloud Console

```
APIs & Services
 → Credentials
   → OAuth Client ID
     → Authorized redirect URIs
```

➡️ **Add production URL**
(Old localhost রাখতে পারো)

Example:

```
http://localhost:8000/accounts/google/login/callback/
https://api.phi-mart.com/accounts/google/login/callback/
```

✔ Multiple allowed

2️⃣ Django allauth — change লাগবে না

allauth automatically callback handle করে
তুমি শুধু **domain change** করলেই হবে

3️⃣ Django Sites Framework (IMPORTANT)

Production domain set করতেই হবে 👇

```
Admin
 → Sites
   → example.com
```

Change to:

```
yourdomain.com
```

⚠️ এটা না করলে error আসবে:

```
No SocialApp matches the request
```

🔐 Production Security Checklist

```
DEBUG = False

SECURE_SSL_REDIRECT = True
SESSION_COOKIE_SECURE = True
CSRF_COOKIE_SECURE = True

ACCOUNT_DEFAULT_HTTP_PROTOCOL = 'https'
```

🚨 Common Production Errors

❌ Redirect URI mismatch
❌ HTTP instead of HTTPS
❌ Wrong Site domain
❌ Missing slash `/`
❌ Subdomain mismatch (`www` vs non-www)

🧠 Pro Tip (Best Practice)

👉 **One OAuth client**

- Multiple redirect URIs

- Dev + Prod together

✅ Final Answer (One Line)

> **হ্যাঁ, production এ গেলে অবশ্যই redirect URL production domain দিয়ে add করতে হবে — Django code change লাগে না, Google Cloud + Sites update করলেই হবে।**

চাও তো আমি:

- তোমার **exact domain অনুযায়ী URL** বানিয়ে দিতে পারি
- **React + allauth prod checklist**
- **DRF JWT + Google prod deploy flow**

বললেই চলি 😄

Excellent question 🔥
এই দুইটা **auth ecosystem** একসাথে দেখে নিলে Django auth পুরো clear হয়ে যায়।

আমি **simple → real-world → decision-making** স্টাইলে তুলনা করছি।

🥊 Djoser vs django-allauth (Clear Difference)

🧠 One-liner first

> **allauth = Django template + session based auth**
> **Djoser = DRF API-only auth (JWT / Token based)**

1️⃣ Core Purpose

Topic	allauth	Djoser
Built for	Django web apps	REST APIs
Auth style	Session / Cookie	Token / JWT
Frontend	Django templates	React / Mobile
Signup/Login	HTML flow	JSON APIs

2️⃣ Authentication Flow

🔹 allauth flow

```
Browser
 → Google OAuth
 → allauth
 → Django session
 → request.user
```

🔹 Djoser flow

```
Frontend (React/Mobile)
 → POST /auth/users/
 → JWT / Token
 → Authorization header
```

3️⃣ Google / Social Login

✅ allauth

- Built-in social auth
- Google, Facebook, GitHub
- Handles:
  - OAuth flow
  - Redirects
  - Signup

👉 **Very easy**

⚠️ Djoser

- ❌ No built-in Google login
- Needs:
  - allauth OR
  - Custom Google token verify

👉 **More control, more work**

4️⃣ API Endpoints

🔹 allauth

- Mostly views, not APIs
- HTML redirects
- Not SPA-friendly

🔹 Djoser (REST)

Example:

```
POST /auth/users/           # signup
POST /auth/jwt/create/     # login
POST /auth/jwt/refresh/
POST /auth/users/reset_password/
```

👉 Clean JSON responses

5️⃣ JWT Support

Feature	allauth	Djoser
JWT	❌	✅
SimpleJWT	❌	✅
Token rotation	❌	✅

6️⃣ Custom User Compatibility

Both support:

- Email-based login
- Custom user model
- Permissions

7️⃣ Email Verification & Password Reset

Feature	allauth	Djoser
Email verify	✅	✅
Reset password	✅	✅
HTML emails	✅	❌
API emails	❌	✅

8️⃣ Security & Control

Area	allauth	Djoser
OAuth security	High	Manual
Token security	❌	High
API attack surface	Low	Needs care
Rate limiting	❌	Needs DRF throttling

9️⃣ When to Use Which? (Decision Table)

✅ Use django-allauth if:

- Django templates
- Server-rendered app
- Session login
- Social login heavy
- Minimal frontend JS

✅ Use Djoser if:

- React / Vue / Mobile
- Pure API backend
- JWT auth
- Microservices
- Mobile apps

🔥 Best Practice (Real-World)

💡 Don't mix randomly

❌ allauth + Djoser (same auth flow)
✅ allauth (social OAuth) + SimpleJWT (custom bridge)
✅ Djoser + custom Google endpoint

🧠 Pro Architecture (Recommended by Pros)

Option-1: Web app

```
Django
 ├─ allauth
 └─ Sessions
```

Option-2: SPA / Mobile

```
React / Mobile
 └─ Google Login
      └─ DRF
           ├─ Custom Google Verify
           └─ SimpleJWT
```

⚠️ Common Beginner Mistake

> "I'll use both allauth and djoser together for everything"

👉 This causes:

- Duplicate users
- Confusing flows
- Security bugs

✅ Final Verdict

> **allauth = browser + OAuth + sessions**
> **Djoser = API + JWT + mobile**

If you want, next I can:

- 🔥 **Djoser + SimpleJWT full setup**
- 🔥 **Replace Djoser with custom auth (more secure)**
- 🔥 **Migrate allauth → API-only**

Just tell me 😎

tutorial link

https://www.youtube.com/watch?v=D1B_BkGHbqs&t=43s